

DeepFWI: Identifying Bug-Sensitive Warnings With Multi-Modal Code-Warning Semantics

Han Liu , Jian Zhang , Cen Zhang , Xiaohan Zhang , Kaixuan Li , Sen Chen , Shang-Wei Lin ,
Yixiang Chen , Xinghua Li , and Yang Liu , *Senior Member, IEEE*

Abstract—Static analysis tools have evolved over time to assist in detecting bugs. However, the excessive false warnings can impede developers’ productivity and confidence in the tools. Previous research efforts have explored learning-based approaches to identify bug warnings. Nevertheless, their coarse granularity, focusing on either long-term warnings or function-level alerts, are insensitive to individual bugs. Also, they rely on manually crafted features or solely on source code semantics, which is inadequate for effective learning. In this paper, we propose DeepFWI, a learning-based approach that identifies bug-sensitive warnings at a fine-grained granularity. Specifically, we design a novel LSTM-based model that captures multi-modal semantics of source code and warnings from automated static analysis tools (ASATs) and highlights their correlations with cross-attention. To tackle the data scarcity of training and evaluation, we collected a large-scale dataset of 280,273 warnings. We conducted extensive experiments on the dataset to evaluate DeepFWI. The experimental results demonstrate the effectiveness of our approach, with an F1-score 67.06% for confirming true warnings in a finer-grained manner, significantly outperforming all baselines. Additionally, to validate the practicality of DeepFWI from the perspective of developers, we applied DeepFWI to four popular open-source projects. Our approach filtered out the vast majority of warnings, while still successfully surfacing 25 true bug-related warnings that were confirmed through manual analysis.

Index Terms—Deep learning, warnings, automated static analysis tool, bug-sensitive.

Received 22 July 2025; revised 23 December 2025 and 23 February 2026; accepted 25 March 2026. Date of publication 30 March 2026; date of current version 1 May 2026. This work was supported by the Fundamental Research Funds for the Central Universities under Grant KG16426001. Recommended for acceptance by P. Borba. (*Corresponding author: Jian Zhang.*)

Han Liu and Sen Chen are with the College of Cryptology and Cyber Science, Nankai University, Tianjin 300350, China (e-mail: hanliu@nankai.edu.cn; senchen@nankai.edu.cn).

Jian Zhang is with the School of Software, Beihang University, Beijing 100191, China (e-mail: zhangj_cs@buaa.edu.cn).

Cen Zhang, Kaixuan Li, and Yang Liu are with Nanyang Technological University, Singapore 639798 (e-mail: cen001@ntu.edu.sg; kaixuan.li@ntu.edu.sg; yangliu@ntu.edu.sg).

Xiaohan Zhang is with the School of Computer Science, Shanghai Jiao Tong University, Shanghai 200240, China (e-mail: xhzhang1@sjtu.edu.cn).

Shang-Wei Lin is with Singapore Institute of Technology, Singapore 828608 (e-mail: shangwei.lin@singaporetech.edu.sg).

Yixiang Chen is with the Shanghai Key Laboratory of Trustworthy Computing, East China Normal University, Shanghai 200062, China (e-mail: yxchen@sei.ecnu.edu.cn).

Xinghua Li is with Xidian University, Xi’an 710126, China (e-mail: xhli1@mail.xidian.edu.cn).

Digital Object Identifier 10.1109/TSE.2026.3678424

I. INTRODUCTION

AUTOMATED static analysis tools (ASATs) are widely adopted in industry for their ability to detect bugs at low cost and without code execution [1]. However, a longstanding challenge remains: ASATs often produce an excessive number of warnings, the majority of which are ultimately irrelevant or false positives. Recent studies show that up to 90% of ASAT warnings may be false positives [2], [3], overwhelming developers and hindering effective bug detection.

To mitigate this flood of warnings, researchers have developed learning-based approaches to prioritize or filter ASAT outputs. Two main directions have emerged: (1) *Ranking* warnings to highlight those most likely to be fixed, and (2) *Identifying* bug-related warnings for focused resolution. However, existing ranking approaches [4], [5], [6], [7] typically operate on groups of warnings and leave open questions about individual warning prioritization. Developers are often left unsure of which specific warnings demand immediate attention. Therefore, we focus this work on the finer-grained problem of identifying which individual warnings are truly related to bugs.

A major thread in prior work is the notion of *actionable warnings*: those ASAT warnings that are likely to be resolved during the software lifecycle [8], [9], [10], [11], [12], [13], [14], [15]. Classical approaches use machine learning classifiers built on manually engineered features of warnings, code, history, and file attributes [12], [13], [14], [15]. For example, Wang et al. identified 23 “Golden Features” that improved actionable warning detection [12]. Yet, recent studies highlight that such approaches are vulnerable to data leakage and duplication, undermining their reliability [1], [2].

An alternative direction leverages deep learning for bug prediction at the function level [16], often using pre-trained Transformer models. While these approaches can boost precision, they may ignore the specific semantics of warning messages and treat all warnings in a function uniformly, making it hard to distinguish between true and false positives on a per-warning basis. More recently, Large Language Models (LLMs) like GPTs have been used via prompt engineering [17], but these approaches are expensive to scale and susceptible to prompt sensitivity and hallucinations.

Despite progress, several key limitations persist in the state of the art:

Limitation 1 Ambiguous Granularity. For actionable warnings detection, existing studies define them over coarse time

frames (e.g., months between commits), making it unclear whether a warning's disappearance truly signals a bug fix or is due to unrelated code changes such as refactoring [1], [2], [9], [12]. As for function-level bug prediction [16], it also fails to distinguish between multiple warnings within the same function.

Limitation 2 Limited Semantics. Feature-based approaches (i.e., [1], [2], [9], [12], [18]) often miss the rich semantics embedded in both code and warning messages, limiting their ability to distinguish true from false positives. Function-level models [16] typically aggregate the entire function into a single representation, which can dilute the specific signals associated with a warning. Additionally, they ignore the rich semantic cues present in warning messages and other relevant code definitions (e.g., class fields), leading to suboptimal performance in identifying bug-sensitive warnings. The LLM-based approach [17] is computationally expensive when applied to large-scale data and prone to irrelevant outputs due to prompt variability and hallucinations.

Limitation 3 Lack of Scale and Generalizability. Most prior work [2], [9], [12] is evaluated on limited datasets, which are often small in scale and lack diversity. This leaves open questions about scalability and generalizability to diverse, real-world projects.

To address these challenges, we introduce the notion of *bug-sensitive warnings*: individual ASAT warnings that appear in a buggy version of code and disappear as a direct result of the corresponding bug-fix commit (formally defined in Section II-D). Unlike generic actionable warnings, which may reflect code changes over long periods and uncertain scope, bug-sensitive warnings are tightly coupled to concrete bug fixes, providing a more precise and timely signal for developers. By focusing on the disappearance of warnings directly tied to bug-fixing commits, we minimize ambiguity caused by unrelated code modifications and improve the granularity of bug detection.

However, automatically identifying bug-sensitive warnings still presents two core challenges:

Challenge 1 Dataset Scarcity. There is no existing large-scale dataset of bug-sensitive warnings suitable for training and evaluation. To bridge this gap, we construct **BSWarnings**, the largest warning-related dataset to date, by mining 607,259 bug-fixing commits across 32,100 open-source projects. Using SpotBugs [19], we systematically pair warnings before and after bug-fix commits, applying rigorous filtering to eliminate spurious cases and deduplication. **BSWarnings** contains 280,273 total warnings, including 20,100 bug-sensitive warnings, validated via random sampling at over 92% accuracy, with a confidence level of 95%. Notably, **BSWarnings** exceeds existing benchmarks by an order of magnitude (10 times) [12]. To achieve this scale, we executed over 1.2 million compilation and analysis tasks, totaling more than 5,000 hours of distributed computation on three servers.

Challenge 2 Multi-Modal Semantics. Bug-sensitive warning identification requires capturing multi modal semantics: the joint representation of several distinct but complementary information sources associated with each warning, for example, the source code context where the warning arises, the natural

language warning message, and structured warning attributes (e.g., warning type, rank). To address this, we propose **DeepFWI**, a novel neural architecture that advances the state of static analysis warning identification in several key ways:

- **Multi-modal, fine-grained representation:** DeepFWI processes both source code and warning messages as first-class, complementary modalities. Unlike previous models that treat warnings in aggregate or ignore warning text, DeepFWI encodes each warning individually, integrating not only the relevant code context but also the structured attributes and rich natural language of warning messages.
- **Cross-attention mechanism for code-warning correlation:** We introduce a cross-attention module that explicitly models the semantic alignment between code slices and warning messages. This allows DeepFWI to focus on the most relevant code regions for each warning, capturing subtle interactions that prior sequence models or simple pooling approaches cannot.
- **Context precision via warning-aware slicing:** DeepFWI incorporates a slicing strategy, extracting code slices tightly linked to each warning. This reduces noise, improves context quality, and ensures the model bases predictions on the most relevant program elements.

In summary, DeepFWI represents a significant advancement in bug-sensitive warnings identification generated by ASATs by combining a large-scale, rigorously validated dataset of bug-sensitive warnings with a novel neural architecture that captures multi-modal semantics at fine granularity. Table I presents a structured comparison of DeepFWI versus state-of-the-art (SOTA) techniques across four critical dimensions: granularity, input modality, and computational cost. As shown, while feature-based approaches [1], [2], [9], [12], [18] rely on coarse-grained manual features and function-level models [16] lack specific warning context, DeepFWI uniquely targets fine-grained code slices. Furthermore, unlike the LLM-based approach [17] which incurs high inference costs and risks hallucinations, DeepFWI leverages a specialized cross-attention mechanism to align code and warning semantics efficiently. This comparison highlights the specific research gap DeepFWI aims to fill: the need for a low-cost, fine-grained, and semantically aware solution.

We comprehensively evaluate DeepFWI against ten baselines, demonstrating its superior precision (at least +9.0%) and F1-score (at least +13.2%) compared to SOTA approaches, including feature-based [1], [2], [9], [12], [18], function-level [16], LLM-based approach [17], and several straightforward solutions [20], [21], [22], [23]. Specifically, in terms of F1-score, DeepFWI outperforms feature-based approaches [1], [2], [9], [12], [18] by 15.81%, the function-level approach [16] by 14.01%, and the LLM-based approach [17] by 53.78%. When evaluated against straightforward baselines, DeepFWI continues to show superiority, surpassing N-gram [23], Transformer [21], GraphCodeBERT [20], and LSTM [22] models by 17.14%, 15.55%, 16.42%, and 13.17%, respectively. Overall, DeepFWI establishes a clear performance advantage, with F1-score margins ranging from 13.2% to 57.6%.

TABLE I
COMPARISON OF DEEPFWI WITH STATE-OF-THE-ART APPROACHES

Approach	Granularity	Input Modality	Cost
Feature-based [1], [2], [9], [12], [18]	Coarse Time Frames with Warnings	Manual Features	Low
Function-level [16]	Function	Implicit	Medium
LLM-based [17]	Function	Code + Prompt	High
DeepFWI (Ours)	Bug-Sensitive Warnings	Multi-modal (Code+Messages+Sliced IR+Attributes)	Low

Unlike standard Transformers which impose strict length limits (e.g., 512 tokens) to maintain efficiency, and modern LLMs which, despite handling long contexts, suffer from substantial computational overhead and slow inference speeds, for instance, the inference time of the modern Qwen2.5-0.5B model is approximately 180 times that of DeepFWI provide an optimal trade-off. They efficiently encode multi-dimensional information and flexibly accommodate longer inputs, preserving critical context while avoiding the heavy memory usage and energy consumption inherent to large-scale Transformer architectures. Our ablation studies (Section IV-D2) further demonstrate that LSTM-based architectures, especially when enhanced with cross-attention, are effective at capturing the sequential and localized dependencies between code slices and warning messages in this task. We also discuss these trade-offs in detail and provide empirical comparisons with state-of-the-art Transformer and LLM baselines.

To assess real-world applicability, we deploy DeepFWI on four popular open-source Java projects (*itext7*, *metadata-extractor*, *poi*, *pdfbox*). While DeepFWI filters out over 92% of potential false alarms—greatly reducing developer workload—it still successfully identifies 25 previously unknown bugs, 4 of which were confirmed and fixed by project maintainers. We also analyze the risk of filtering out true positives and discuss its practical implications.

In summary, this paper makes the following contributions:

- We present **BSWarnings**, a rigorously validated dataset of 280,273 warnings, including 20,100 bug-sensitive warnings, mined from over 600K bug-fixing commits.
- We propose **DeepFWI**, a novel neural model for fine-grained, warning-level bug identification, integrating multi-modal semantics from source code and warning messages using cross-attention.
- We provide an extensive evaluation that DeepFWI outperforms ten SOTA baselines and generalizes to new projects, supporting practical adoption.
- All data and code are publicly released to future research [24].

II. PRELIMINARY

A. Automated Static Analysis Tools

Automated Static Analysis Tools (ASATs), popular in software development, are designed to analyze code for bugs and code smells without execution. Notable examples include SonarQube [25], SpotBugs [19] (formerly FindBugs [26]), and Infer [27]. In particular, SpotBugs [19] is an open-source static

code analysis tool utilized in software development for bug detection in Java code. As the direct successor to FindBugs [26], it is among the most highly regarded tools in its domain. SpotBugs carries forward the same fundamental technology but with continuous support and improvements.

While ASATs are effective in bug detection during software development stages [28], [29], [30], [31], they depend on human-made rules or established patterns to identify known errors. The tendency of ASATs to generate an overwhelming number of warnings to cover potential bugs limits their usefulness in large software projects [3], [32], [33], [34]. To mitigate the occurrence of false alarms in ASATs, various methodologies have been proposed to identify truly beneficial warnings. One focal point of these studies is actionable warnings, i.e., warnings that developers would rectify over time. Such studies contemplate multiple features of the warnings and the accompanying code [12], [13], [14], [15]. However, there exist issues like data leakage and duplication that severely undermine the reliable identification of truly actionable warnings [2]. Moreover, actionable warnings may not reflect the remediation of bugs since it is based on the long span of commit history where the source code evolves greatly. Alternatively, Kharkar et al. utilized a learning approach to minimize false positives [16]. They apply CodeBERTa as the foundation model to identify the function of related warnings whether there is a bug and show an improved accuracy. Clearly, this approach operates at a coarse granularity that considers only the entire function.

B. Actionable Warnings

In the context of static analysis, an actionable warning refers to a warning reported by an ASAT that is deemed relevant and meaningful for developers to address during software maintenance [1], [2], [9], [12]. Unlike generic warnings that may include minor code style suggestions or low-impact issues, actionable warnings highlight potential defects or code problems that are likely to be fixed by developers over the course of normal development.

Formally, a warning is considered actionable if it corresponds to an issue that is subsequently resolved—typically within a defined time frame (e.g., 3 months)—through code changes such as bug fixes or targeted improvements. Actionable warnings are thus distinguished by their practical impact: they prompt corrective action and have a higher likelihood of being addressed compared to non-actionable or spurious warnings. Prior work [2], [9], [12] has shown that identifying actionable warnings can significantly reduce developer workload by filtering out noise and focusing attention on the most critical alerts.

```

1 private static String sendRequest(...) throws
  ↳ Exception{
2     ...
3     if (HttpMethods.POST.equals(method)) {
4         ...
5 -     connection.setRequestProperty(
  ↳ REQUEST_HEADER_CONTENT_LENGTH, Integer.toString(
  ↳ data.getBytes().length));
6 + //connection.setRequestProperty(
  ↳ REQUEST_HEADER_CONTENT_LENGTH, Integer.toString(
  ↳ data.getBytes().length));
7     ...
8     }
9     ...
10    InputStream is = connection.getInputStream();
11    BufferedReader rd = new BufferedReader(new
  ↳ InputStreamReader(is));
12    ...
13 }

```

Fig. 1. Simplified code snippets from DongTai-agent-java (Commit 2822534) that motivate our work.

C. Motivating Example

When dealing with vast and numerous projects, developers often find themselves overwhelmingly burdened by a large number of warnings reported by ASATs [3]. Hence, it is necessary to reduce the false alarms of the warnings. However, existing work can easily encounter problems due to the pre-defined features or the coarse granularity. They cannot effectively distinguish the individual warnings, which may result in mislabeling.

The following commit¹ is from the project DongTai-agent-java of organization HXSecurity. As shown in Fig. 1, it fixed a bug caused by `data.getBytes()` when the platform's default charset is not compatible with the data being sent. Since the `CONTENT_LENGTH` header is not always necessary in a POST request, the fixed version has commented Line 5 out, and let the request automatically handle the content length.

SpotBugs reported a warning related to reliance on default encoding at line 5, which disappeared in the revised version. As such, it qualifies as a bug-sensitive warning. Meanwhile, SpotBugs issued a similar warning at line 10, involving the same function. Here, however, the `InputStream (is)` involves raw byte data obtained from the server response, which is then decoded into characters using the default charset. This operation actually functions correctly because the server's response is encoded using a charset compatible with the default. Accordingly, we identified this as a bug-insensitive warning. Given that the two warnings originated from the same function, existing function-level learning-based warning verification approaches and feature-based approaches would consider the same input, potentially leading to mislabeling. Hence, to distinguish these bug-sensitive warnings from the bug-insensitive warnings and lower human effort when finding bugs using ASATs, a finer granularity approach for the warning identification is imperative.

D. Problem Definition

Our task is to facilitate developers to identify the bug-sensitive warnings, allowing us to reduce the burden

on developers of sifting through potentially large numbers of irrelevant warnings, and instead focus their efforts on addressing bugs that have been identified with high confidence. We formally define the task as follows.

In the context of static analysis, where the process maps a codebase $\mathcal{C}$ to a set of warnings $\mathcal{W}$ (i.e., $SA: \mathcal{C} \rightarrow \mathcal{W}$), a bug-sensitive warning is defined to rigorously capture warnings causally linked to actual defects. Formally, given a historical dataset $\mathcal{H}$ of validated bug-fix pairs (C_b, C_f) , where C_b represents buggy code and C_f represents fixed code, we define a set of bug-sensitive warnings $W_b \subseteq \mathcal{W}$ as follows:

A warning $w \in \mathcal{W}$ is classified as *bug-sensitive* if:

$$w \in W_b \iff \exists (C_b, C_f) \in \mathcal{H} : \begin{cases} w \in SA(C_b) \wedge w \notin SA(C_f), \\ L(w, C_b) \cap \Delta(C_b, C_f) \neq \emptyset. \end{cases} \quad (1)$$

where:

- $L(w, C_b)$ maps w to code locations (i.e., lines) in $\mathcal{C}$ where w is triggered,
- $\Delta(C_b, C_f)$ denotes code modifications between buggy code C_b and fixed code C_f .

Given a codebase $\mathcal{C} = \{c_1, c_2, \dots, c_N\}$ and its associated static analysis warnings $\mathcal{W} = \{w_1, w_2, \dots, w_N\}$, the task is to classify each warning $w_j \in \mathcal{W}$ as either bug-sensitive or bug-insensitive. Formally, this is framed as a supervised learning problem where a classifier F learns the discriminative mapping:

$$F: \mathcal{C} \times \mathcal{W} \rightarrow \{0, 1\}, \quad \text{such that } F(c_i, w_j) = y_{ij} \quad (2)$$

Here, $y_{ij} \in \{0, 1\}$ is a binary label where $y_{ij} = 1$ indicates that warning w_j in code snippet c_i is bug-sensitive, and $y_{ij} = 0$ otherwise.

Given the variations across programming paradigms and languages, our approach focuses on Java and SpotBugs following the existing research [9], [12], [16]. To clarify the terminology, we use “function” to describe our methodology to present a comparable discussion with existing work [9], [12], [16] using “function”, while “method” is used in some Java-specific contexts to refer to Java methods.

III. METHODOLOGY

A. Overview

In this section, we introduce DeepFWI, a deep learning-based (DL-based) approach to identify warnings of bugs for ASATs. Different from existing feature-based [9], [12] and learning-based approaches [16], DeepFWI is designed to automatically capture more comprehensive semantic information from the code, warnings, and their interaction, applying it across a broader scope and with finer granularity warning identification.

B. Data Collection

Considering that current datasets of warnings, such as [12], are limited by coarse granularity, either in the dimension of time (i.e., actionable warnings) or code (i.e., function level), they do not align with our requirement for bug-sensitive warnings

¹ <https://github.com/HXSecurity/DongTai-agent-java/commit/2822534>

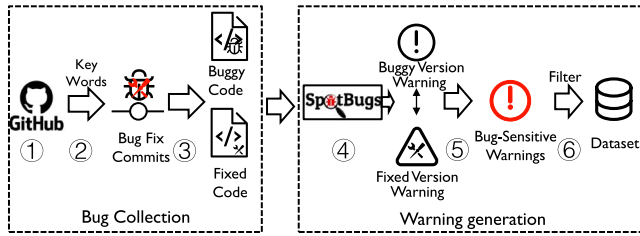


Fig. 2. The pipeline for collecting bug-fix commits and generating bug-sensitive warning datasets.

identification, which demands a finer granularity. In addition, the scale of the existing datasets is limited to the specific projects. Hence, to construct an automated approach that identifies warnings of bugs via deep learning, our initial step involves creating a dataset, BSWarnings, gathering a large number of bug-sensitive warnings. The overall data collection process is illustrated in Fig. 2, which consists of six main steps:

- ① **Project Selection:** We begin by searching for as many Java projects as possible. To ensure dataset quality, we filtered projects by selecting those in the top 0.3% (about 32,100 Java projects) based on GitHub stars to ensure we focused on popular and actively maintained repositories. The 0.3% threshold was chosen to balance the need for a manageable dataset size while still capturing a diverse range of high-quality projects. Below this threshold, we observed a significant drop in project activity and maintenance, which could affect the reliability of our analysis.
- ② **Bug-Fixing Commit Identification:** We identify bug-fixing commits within these projects by applying keyword filtering to commit messages, following prior work [35]. Specifically, to be identified as such, a commit message must contain the word “fix” along with other bug-related words such as “bug”, “defect”, “error”, “fault”, “issue”, or bug classification words like “null pointer dereference”, “resource leak” etc. Using this filtering process, we collected a total of 607,259 related commits.
- ③ **Buggy and Fixed Code Extraction:** For each identified commit, we clone the project and obtain both the buggy (pre-fix) and fixed (post-fix) versions of the code.
- ④ **Warning Generation:** We compile both code versions and run SpotBugs to generate static analysis warnings for each.
- ⑤ **Bug-Sensitive Warning Identification:** We compare the warnings generated from the relevant files in both the buggy and fixed versions of the commits. If the warning disappears in the fixed version, we mark it as a bug-sensitive warning. For deleted functions, if the corresponding functions are missing in the fixed version, then by our definition, $\Delta(C_b, C_f)$ is empty. Consequently, we do not consider the warning to be bug-sensitive—instead, we designate it as bug-insensitive because it is not correlated with the specific bug. Similarly, newly added functions are excluded from consideration. Since there is no corresponding buggy version available for comparison, the warnings generated by these newly added functions fail to

meet the second condition, which requires that $L(w, C_b) \cap \Delta(C_b, C_f)$ be non-empty.

- ⑥ **Deduplication and Final Dataset Construction:** Finally, we remove duplicate and bug-insensitive warnings, resulting in BSWarnings, which contains 280,273 warnings, including 20,100 bug-sensitive warnings.

Notably, BSWarnings represents the largest collection of warning-related data to date, exceeding previous benchmarks by an order of magnitude (10 times) [12]. To achieve this scale, we executed over 1.2 million compilation and analysis tasks, totaling more than 5,000 hours of continuous distributed computation across three high-performance CPU servers, each equipped with dual Intel x64 48-core CPUs and 256 GB of RAM with Ubuntu 18.04 Operating System.

C. Fine-Grained Warning Identification

Now we present our DL-based approach to bug-sensitive warning identification. Our approach includes four main components, i.e., code context extraction, warning information extraction, attention encoder model, and feature fusion. The framework of our approach is shown in Fig. 3. Firstly, we extract the code context and warning information from the target class reported by specific warnings. Then, based on the code context and warning information, we design an attention encoder model on them to obtain a unified vector representation. The vector is finally used to determine whether the warning is bug-sensitive or not.

1) **Code Context Extraction (①–⑤ in Fig. 3):** The step of code context extraction consists of three key parts: warning function extraction (① and ③ in Fig. 3), field of class extraction (① and ② in Fig. 3), and warning-aware slicing (④ and ⑤ in Fig. 3). The warnings provide the potential bugs present in a specific class and function with the specific lines. Due to the multi-modal sources involved in one warning, such as code, messages, and rules, it becomes necessary to extract sufficient and precise context for the model to effectively identify the warnings. Considering that the length of input for a neural model is typically limited, our context extraction of the code is within the target class.

Warning Function Extraction (① and ③ in Fig. 3): To provide more related information for the network, we first focus on the function (i.e., method) on which the specific warning is reported. As described before, each warning points to the class where the bugs may exist, so we locate the file for further extraction. However, due to the likelihood that the warning is missing the function name, in addition, there may be function overloading in the specific class, we extract the function with the following criteria: (1) If the warning is reported with lines, we use the lines to locate the unique function. If the lines are not a specific function, we recognize the context of this warning is the whole class. (2) If the warning is reported with a function name, we locate all the functions with the same name as the context. Otherwise, we use the class as the context of the warning. In this step, we use ANTLR4 [36] to build the abstract syntax tree (AST) for the buggy file and then obtain the function. If we cannot find the function corresponding to

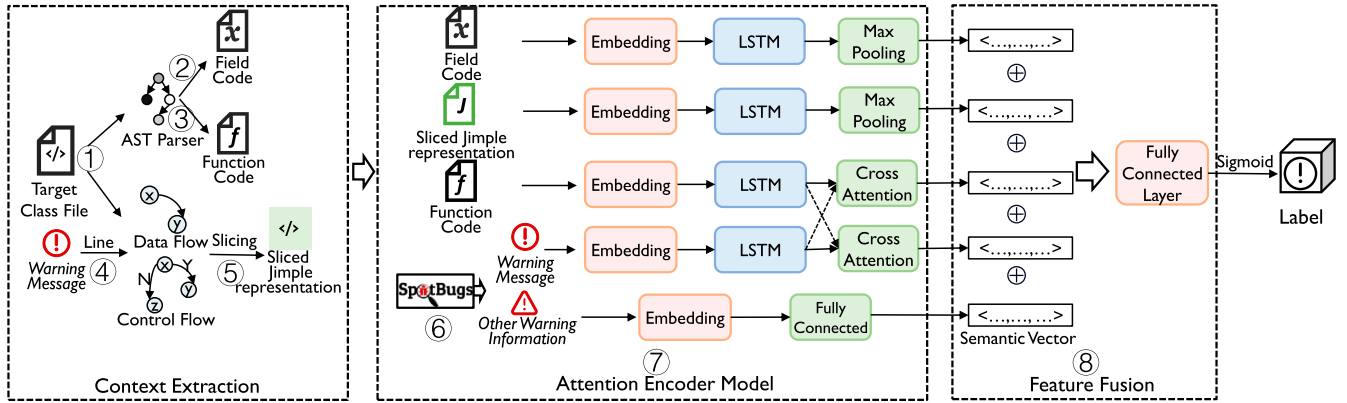


Fig. 3. The architecture of DeepFWI: attention-based warning identification model.

the line number, or if there is no line number available, we use the entire class content to fill in this input.

Field of Class Extraction (① and ② in Fig. 3): In addition, considering the characteristics of the Java language, Java fields, i.e., member variables in a class [37], which are declared inside a class, but outside any method, constructor, or block, can be related to all functions. They are object-specific and have potentially different values for different objects of the same class. Fields can be categorized as public, private, or protected, which are visibility modifiers that define how and from where these fields can be accessed. This external context might also hold key clues related to various potential errors. For instance, issues like incorrect field initialization could lead to unexpected application behavior, while null pointer exceptions might arise from fields not being properly instantiated. Therefore, we extract the field code in the class to provide more information for the model. In this work, field code refers to the segment of source code that is dedicated to declaring class-level fields, including field names, modifiers, types, and initial values or assignments. For example, in a declaration like `private int counter = 0;`, the field code would include the modifier (private), the type (int), the field name (counter), and the initializer (0). To extract the field code, we start with the AST constructed by the code file. Then, we locate the root of the class and traverse the class. If the node is the definition of variables and does not belong to any function method, we classify it as a part of the fields. Finally, we obtain the field codes in the related class.

Warning-Aware Slicing (④ and ⑤ in Fig. 3): Although we have pinpointed the exact function where the warning was issued, the data fed into our model might be noisy, e.g., the data dependency does not influence the specific bugs, or arbitrary names of local variables, hindering its ability to concentrate on the specific buggy lines and their directly related fragments. To mitigate this, we introduce a warning-aware slicing on the function. The slicing aims to (1) isolate all relevant dependencies, thereby ensuring the model's focus remains strictly on the essential elements related to the bug; (2) restructure the program and unify variable names.

Specifically, we set the lines that the warning reported as the slice criteria, and do the backward and forward slice according

to the slice criteria. We first use Soot [38] as the analysis framework and construct the program dependency graph (PDG) of the project. For the backward slice, we focus on the variable read in the slice criteria and find all nodes in the PDG that are directly or indirectly dependent on the specified program point starting from the slice criteria. As for the forward slice, we focus on the variable written in the slice criteria and still find all nodes that are directly or indirectly dependent. Note that our slices are confined to the function's scope, as interactions between the function and class are intricate. These interactions could involve semantics associated with other warnings, thereby leading to the potential misguidance of our model. Finally, we obtain the sliced function code originally in the form of the intermediate representation (IR) in Soot [38], i.e., Jimple. Jimple is a typed 3-address IR. It provides a more standardized and simplified view of observing program behavior, which can assist the model in understanding the semantics of complex statements (e.g., multiple calls) and excluding the interference from the semantics of some local variables.

As shown in the Fig. 4, the code snippet is a Jimple representation of the Java code Line 5 in Fig. 1. The original Java code has been converted to three typed 3-address statements. The statements break down complex expressions and statements into simpler ones that perform a single operation and have at most one side effect. For example, the original code Line 5, involves many assignments (e.g., `lengthof` corresponds to the `.length` property used in the original Java code) and function invoke (e.g., `getBytes`, `toString`, and `setRequestProperty`). The Jimple code divided the code in Line 5 into several different statements. In addition, the intermediate variables are simplified (e.g., `stack61`). Hence, by unifying the local variables, splitting the complex semantics, Jimple representation can guide the model more effectively to locate the bugs from the given function and messages. Again, if there is no corresponding function, we use the entire class's Jimple as a filler.

2) Warning Information Extraction (⑥ in Fig. 3): Besides the code context we mentioned, some warning information needs to be provided to the model. First of all, we collect the MESSAGES of each warning, which include the details of

```

1 $stack61 = virtualinvoke data.<java.lang.String: byte[]
   ↳ getBytes()>()
2 $stack62 = lengthof $stack61
3 $stack63 = staticinvoke <java.lang.Integer: java.lang.
   ↳ String toString(int)>($stack62)
4 virtualinvoke connection.<java.net.HttpURLConnection:
   ↳ void setRequestProperty(java.lang.String, java.
   ↳ lang.String)>("Content-Length", $stack63)

```

Fig. 4. A simplified example of Jimple representation of Java code converting from Fig. 1.

TABLE II
THE DETAILS OF THE WARNING INFORMATION IN SPOTBUGS

Information	Description
MESSAGE	Error message generated by SpotBugs for warning description.
RULE	The scanning rule for triggering the warning.
CATEGORY	The category of the rule belongs to (e.g., CORRECTNESS).
RANK	A specified bug severity rank of warning.
CONFIDENCE	A particular bug confidence of warning reported.

the potential bugs and can serve as a filter for the model to identify the buggy semantics. Then, we gather the RULE and CATEGORY attributes for the model to distinguish different types of potential bugs. Finally, we also collect the RANK and CONFIDENCE of the warning for the model to judge the risk of potential bugs and possible concerns of the ASATs. The details of the warning attributes used are shown in Table II.

3) *Attentional Encoder Model* (⑦ in Fig. 3): While Transformer-based architectures and LLMs have demonstrated significant success in a variety of code and natural language understanding tasks, there are important reasons for employing an LSTM-based model in our context. First, our task requires encoding multi-dimensional information for a large number of warnings in real-world projects. Utilizing LLMs or Transformer-based models for every warning would be computationally expensive and inefficient, especially when the warning volume is high. Second, most pretrained Transformer models impose strict input length limits (typically 512 tokens), which can lead to the loss of critical context when encoding larger code regions or multiple warnings. In contrast, LSTMs offer both efficient computation and the ability to flexibly handle longer or variable-length sequences, making them a practical and scalable choice for fine-grained bug-sensitive warning identification.

While modern LLMs and extended-context embedding models (e.g., Qwen, GPT-4) support extensive input sequences (8k-32k+ tokens), applying them to the domain of static analysis creates a scalability bottleneck. ASATs frequently generate tens of thousands of warnings; processing such a massive volume with heavy-weight LLMs incurs prohibitive computational costs and latency, making them unsuitable for real-time Continuous Integration (CI) pipelines.

Hence, our approach applies a relatively lightweight model. This enables us to explore dedicated strategies for capturing more precise semantics. In particular, we design an attention encoder model, including five embedding layers, four LSTM

models, two max-pooling layers, two cross-attention modules and two fully connected layers. The five embedding layers and four LSTM models separately encode the function code, the field code, the sliced IR, the warning message, and SpotBugs warning attributes (i.e., RULE, CATEGORY, RANK, and CONFIDENCE features). This rationale for choosing LSTM is supported by prior research [39], [40], [41], which has demonstrated the effectiveness in similar contexts. Given that warning messages typically reflect the semantics embedded within the code, we independently perform cross-attention between the function code and its corresponding warning message to enhance semantic alignment. After encoding the field code, the sliced IR, and warning attributes through max pooling and full connection. Max pooling was chosen because it effectively extracts the most prominent features from local regions of the encoded sequences—filtering out noise and reducing dimensionality—to yield robust representations for further processing. The fully connected layer is utilized to integrate the diverse contextual cues into a unified high-level representation, enabling the model to capture complex interactions and accurately classify warnings as bug-sensitive or bug-insensitive. Finally, we obtain 5 semantic vectors.

Formally, let f denote the sequence of words $\{s_1, s_2, \dots, s_n\}$ in a specific function, and an embedding layer is applied to initialize the sequence by vectors:

$$f_i = W_e^\top s_i, i \in [1, n], \quad (3)$$

where n is the length of the sequence (i.e. function f), W_e is the embedding matrix. Then, we employ a Long Short-Term Memory (LSTM) [42] to encode the sequence of vectors $F = \{f_1, f_2, \dots, f_n\}$ into hidden states $H_f = \{h_1, h_2, \dots, h_n\}$. To capture the semantics before and after the current position, we further use a Bidirectional LSTM (Bi-LSTM) [22] on it. In this step, we formalized the result is

$$h_j = Bi-LSTM(f_j, h_{j-1}) \quad (4)$$

Similarly, we obtain the hidden states of field code, sliced IR, and warning message by three new Bi-LSTMs. For ease of illustration, we mark them as H_{fe} , H_J and H_{wm} respectively.

After encoding all the code and message, we design a cross-attention mechanism on the vector representations of function code and warning message. The two cross-attention blocks in Fig. 3 (center) implement this bidirectional conditioning between the function-code sequence and the warning-message sequence. Intuitively, cross-attention allows the model to answer questions such as “given this warning message, which parts of the method are most relevant?” and “given this method, which words in the warning message matter most?”. To this end, we compute an attention-weighted summary of the warning message conditioned on the encoded method, and symmetrically an attention-weighted summary of the method conditioned on the warning message. These two summaries capture the most informative code-warning interactions for classification.

Formally, given the hidden states of function code are $H_f = \{h_{f_1}, h_{f_2}, \dots, h_{f_n}\}$, we first calculate the query vector q_f by max pooling H_f as follows:

$$q_f = \text{maxpooling}(H_f) \quad (5)$$

The context vector V_f is computed using the following formula:

$$V_f = \sum_{i=1}^n \alpha_i \cdot h_{m_i} \quad (6)$$

where n is the number of hidden states in the function code, and α_i is the attention weight for each hidden state h_{m_i} , calculated as follows:

$$\alpha_i = \frac{\exp(q_s \cdot h_{m_i})}{\sum_{j=1}^n \exp(q_s \cdot h_{m_j})} \quad (7)$$

The context vector V_f is the weighted sum of the hidden states based on their attention weights, allowing the model to focus on the most relevant semantics of the warning message. For simplicity, we notate the process as $V_f = \text{Attention}(q_f, H_m)$. In this way, we depict the relationships between the whole semantics of the sliced IR code and the words in the warning message.

Following the process, we can obtain the context vector of the message V_m , which captures the relationships within the sliced function through a query vector q_m . That is, $V_m = \text{Attention}(q_m, H_f)$. By computing the attention weights in a mutual manner, we can better acquire the correlations between the sliced code and the warning message.

For the field code vector, H_{fc} , we apply a max pooling on it, to achieve a semantics vector by $V_{fc} = \text{maxpooling}(h_{fc})$. Similarly, for sliced IR, we obtain a semantics vector V_J by max pooling. Besides, to leverage other attributes of warnings, we encode these attributes separately as vectors. Suppose that the t -th attributes is a_t , where W_{e_t} is the weight of the attributes t , we embed it into $x_t = W_{e_t}^\top a_t$, $t \in [1, 4]$ and then apply a fully connected layer on it, i.e., $V_{a_t} = W_a^\top x_t + b_t$. These encoded attributes are then combined into an integrated feature vector, denoted as V_a . As a result, we obtain 5 vectors for the semantics from different aspects.

4) *Feature Fusion* (⊗ in Fig. 3): Based on the above calculations, we have obtained five semantic vectors: field code vector V_{fc} , function code vector V_f , sliced IR representation vector V_J , warning message vector V_{wm} , and other warning attributes vector V_a . Further, we need to integrate these four vectors to obtain a single vector by concatenating the vector. The calculation is as follows:

$$V = V_{fc} \oplus V_f \oplus V_J \oplus V_{wm} \oplus V_a \quad (8)$$

Finally, we employ a fully connected layer on the vector V to get the final vector V_l .

5) *Focal Loss*: Since ASAT will explode with a large number of warnings, the data we collected also face a situation where bug-sensitive warnings and bug-insensitive warnings samples are extremely imbalanced (i.e., close to 1:15). Therefore, during the training process, we used focal loss [43] to handle this situation. The Focal Loss function handles the issue of class imbalance in training exercises such as object detection. It modifies the cross entropy loss with a modulating term to concentrate on challenging instances of misclassification. The Focal Loss function is essentially a cross-entropy loss that is scaled dynamically, where the scaling component reduces to

zero as the certainty in the accurate class rises. Hence, training our approach is to minimize the loss function:

$$FL(p_t) = -\alpha_t(1 - p_t)^\gamma \log(p_t) \quad (9)$$

$$p_t = \exp\left(\frac{1}{N} \sum_i -[y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]\right) \quad (10)$$

where α and γ is the tunable focusing parameter, y_i is the label of i , p_i is the probability of predicting a bug-sensitive warning. To train the model, we apply the sigmoid activation function as the last linear layer for warning identification, and obtain the vector L , which represents the possibility that the corresponding warning is bug-sensitive. After optimizing all the parameters, the models are trained. They output probabilities L for different labels. Therefore, we obtain the prediction using the following equation:

$$\text{Prediction} = \begin{cases} \text{bug-sensitive,} & L > \theta \\ \text{bug-insensitive,} & L \leq \theta \end{cases} \quad (11)$$

where θ is the threshold.

IV. EVALUATION

A. Dataset Evaluation

Since existing datasets are not applicable to our fine-grained warning identification task, we collected BSWarnings with 280,273 warnings according to our approach. However, the quality of the dataset may still affect the validity of our approach. Although we have employed a keyword filter and warning comparison approach to avoid invalid warnings, it is possible to introduce some warnings with incorrect labels. Hence, we conduct a manual evaluation of the dataset. We used stratified random sampling [44] to ensure representation across different warning types. To determine the appropriate sample size, we applied the formula for estimating proportions with a 95% confidence level and a 5% margin of error. Additionally, considering the class imbalance in BSWarnings, we allocated samples proportionally to each class to accurately reflect their distribution. Finally, we sampled 27 bug-sensitive warnings and 357 bug-insensitive warnings from BSWarnings to evaluate the labeling accuracy.

Then, three experienced senior developers from our industry partner, each with frequent experience using SpotBugs, independently evaluated the sampled warnings. For every sampled instance, evaluators were provided with: (i) the SpotBugs warning message and attributes, (ii) the buggy and fixed versions of the affected file, and (iii) the corresponding bug-fix diff. The developers were instructed to determine the correctness of the assigned labels based on the following criteria: (i) For bug-sensitive warnings, the developers checked whether the warning's location overlapped with code changes that directly addressed the defect described by the bug-fix commit, according to our formal definition in Section II-D. (ii) For bug-insensitive warnings, they assessed whether the warning was unrelated to the bug-fix (e.g., disappeared due to unrelated refactoring or

remained unchanged) and whether it might nonetheless indicate another defect.

We first asked the three senior developers to individually assess the sampled data. After the individual assessments (the Kappa score in this process is 0.728, indicating substantial agreement [45]), the three evaluators discussed all cases and resolved disagreements by consensus, producing a final ground-truth label for each sampled warning. Ultimately, the three evaluators reported that among the 27 bug-sensitive warnings, 2 were mislabeled, and among the 357 bug-insensitive warnings, 13 were mislabeled. In conclusion, BSWarnings has an accuracy rate of 92.60% on bug-sensitive warnings and 96.36% on bug-insensitive warnings with a confidence of 95%.

B. Experiment Setup

We carried out the experiments on BSWarnings originally consisting of 280,273 warnings. For the purposes of training, validation, and testing, we divided BSWarnings in an 8:1:1 ratio. Additionally, we applied our trained model to four open-source projects to evaluate the performance of DeepFWI in real-world scenarios.

We implement DeepFWI based on pytorch 1.8.0. In the training process, we set the vocabulary size to 100,000, the embedding size to 512, and the hidden state size of LSTM to 512. We set the batch size to 64 and adopt the Adam [46] as the optimizer. The learning rate is $5e-5$ with a decay when the F1-score for the bug-sensitive samples has stopped improving. The hyperparameters of focal loss α and γ are set to 0.05, 2, respectively. All aforementioned hyper-parameters are chosen from a range of alternatives, based on their performance on the validation set. All the experiments are conducted on one Ubuntu 18.04 server with 48-core CPU, 256GB RAM, and two NVIDIA RTX 3090 GPUs with 24GB of memory.

C. Baselines

DeepFWI represents the first of its kind in identifying ASAT warnings at a fine-grained warning level based on deep learning models. To demonstrate its effectiveness, we conducted a comparative analysis with existing representative learning-based approaches. These approaches include: “GHOST2 [1], [2], [9], [12]”, “DeepInferEnhance [16]”, “FRUGAL [18]”, and “LLM4SA [17]”. Additionally, we also implemented four straightforward solutions including “GraphCodeBERT model [20]”, “transformer model [21]”, “LSTM model [22], [42]”, and “N-gram model [23]”. In our comparisons, we adopted the default settings for the above approaches unless specified otherwise.

1) SOTA Approaches:

GHOST2: Wang et al. and Yang et al. conducted a study on existing features to predict actionable warnings and identified key attributes as “23 golden features” [9], [12]. The features include 8 categories, involving file, history, code, and warnings. Based on this, Kang and Yedida et al. continued the research on golden features and proposed GHOST2 to achieve the best performance [1], [2]. Hence, we followed the same settings of Kang and Yedida et al. approaches to replicate this.

FRUGAL: In addition to the aforementioned supervised learning approaches, there is a semi-supervised learning approach, FRUGAL, for actionable warning identification. FRUGAL, was introduced by Tu et al. [18], and achieved nearly the same result with supervised learning in actionable warning identification. For this baseline, we use the code provided in their official repository.

DeepInferEnhance: This approach [16], introduced by Kharkar et al., employs a pre-trained model CodeBERTa to determine if code functions flagged by warnings contain bugs. We specifically chose CodeBERT [47] for its superior performance, and establish it as a baseline for our study. In addition, as our work takes SpotBugs as input and carries out fine-grained warning prediction, we also made some modifications to the DeepInferEnhance model. We input both the warning message and code simultaneously and used a fully connected layer to fuse these two aspects, making it our baseline.

LLM4SA: Wen et al. [17] conducted a study using LLMs to investigate LLMs’ ability to identify ASAT warnings. They developed the LLM4SA framework and evaluated it in three C++ ASATs. For this baseline, we followed the same setting in the paper and used the GPT-3.5 version of LLM4SA which outperformed Llama-2 in the paper. Additionally, we also implemented LLM4SA using DeepSeek-V3 [48] as another baseline, which is an open-source model with lower cost.

2) Straightforward Solutions:

GraphCodeBERT Model: In addition to the aforementioned CodeBERT model in DeepInferEnhance, we also used another pre-trained model GraphCodeBERT [20]. This model incorporates data flow into the foundation of CodeBERT, resulting in superior semantic understanding compared to CodeBERT. There are some studies that use GraphCodeBERT to detect bugs and vulnerabilities [49], [50]. For this baseline, we used the same settings as CodeBERT. The input lengths of the code and the message are 512 and 32, respectively.

Transformer Model We additionally employ a common transformer [21] to replace the pre-trained model, which can effectively extract semantic information from code. There are also some studies [51], [52] to predict bugs and vulnerabilities based on it. Samely, we concatenate the code and message together and use a fully connected layer to obtain the label. The input size of the code and message are 512 and 32. The hidden dimension of the transformer is 1024. The numbers of heads and encoders are 8 and 6, respectively.

LSTM Model: In this model, we use LSTM [22], [42] as the encoder to replace the transformer. We encode the code and message separately and concatenate them. The hidden dimension of LSTM is 1024, and the n layers of LSTM is 1. Based on this, we apply a fully connected layer to obtain the label.

N-gram Model: Lastly, we use an N-gram [23] model as a baseline. N-gram are widely used in natural language processing and computational linguistics. We concatenate the code and message together as input to verify the validity of the warning ($N = 3$). We built a vocabulary using TF-IDF [53] and applied a linear SVM as the machine learning approach, ultimately obtaining our baseline.

TABLE III
RESULTS COMPARISON WITH DIFFERENT BASELINES FOR WARNING IDENTIFICATION (DEEPINFERENHANCEWM REFERS TO DEEPINFERENHANCE WITH WARNING MESSAGES, LLM4SA_GPT REFERS TO LLM4SA USING GPT-3.5-TURBO MODEL, AND LLM4SA_DEEPSEEK REFERS TO LLM4SA USING DEEPSEEK-V3 MODEL)

Methods	Precision (%)	Recall (%)	F1-Score (%)
GHOST2	51.47	51.04	51.25
DeepInferEnhance	49.59	44.89	47.12
DeepInferEnhance_WM	59.51	47.86	53.05
FRUGAL	5.62	85.53	10.36
LLM4SA_GPT	7.21	83.47	13.28
LLM4SA_DeepSeek	7.03	32.25	11.55
GraphCodeBERT	59.30	47.00	52.44
Transformer	51.03	51.99	51.51
LSTM	59.45	49.28	53.89
N-grams	66.49	39.96	49.92
DeepFWI	75.52	60.31	67.06

To ensure a fair comparison, we adopted different training strategies tailored to the model architectures. DeepFWI, Transformer Model, and LSTM model were trained from scratch on the BSWarnings dataset, initialized with random weights. For the pre-trained Transformer-based baselines (CodeBERT and GraphCodeBERT), we initialized them with their public pre-trained checkpoints and performed full fine-tuning on the BSWarnings dataset to adapt their representations to the target task.

D. Results and Discussion

We present the results of our experiments and delve into the analysis by answering the following research questions (RQs).

1) *RQ1: How effective is DeepFWI compared to the baseline models?*: Table III presents a comparative analysis of DeepFWI and various baseline models in terms of their ability to identify bug-sensitive warnings using metrics precision, recall, and F1-score. The dashed line separates the SOTA approaches and straightforward solutions. The highest values across all columns are highlighted in bold. As depicted in Table III, DeepFWI effectively reduces the cost of manually inspecting warnings. It can identify 60.31% of bug-sensitive warnings with 75.52% precision. Overall, DeepFWI performs with 67.06% F1-score in the warning identification at a finer granularity.

Compared to other approaches, DeepFWI generally outperforms the eight baseline models. The “GHOST2” approach, which previously performed well, appears to be less effective when dealing with more fine-grained bug-sensitive warnings, with a 51.25% F1-score. The primary reason is that fine-grained bug warning detection demands the capture of extensive semantic information. However, the approach in question relies on a set of fixed features, which are insufficient to cover the diversity and complexity of bug semantics. The manually extracted semantics may provide identical features in certain scenarios that are challenging to distinguish, thereby making it difficult for the classifier to differentiate them and resulting in a notably low recall and precision. Fig. 1 is a typical example where “GHOST2” performs failed in the warning identification. SpotBugs reported a warning in the function `sendRequest` of lines 5 and 10

with the same warning type. Hence, the warning characteristic features and code features can not distinguish them. Besides, the codes are both introduced in the first commit of the project, the features of warning history also failed in finding differing aspects of them. Consequently, the classifier recognizes the two warnings are the same warnings and fails to identify these warnings.

Despite DeepInferEnhance undergoing pre-training on a large code corpus, it fails to meet the characteristics of this challenging task. A notable limitation arises when the input lacks a warning message; the original model struggles to accurately differentiate between various warnings within the same function. This leads to a modest F1-score of 47.12%. In contrast, incorporating warning messages into the analysis improves the F1-score by 5.93%. As for another pre-trained model, GraphCodeBERT, which have shown better performance than CodeBERT in other tasks, also struggles to accurately identify bug-sensitive warnings, with only 52.44% F1-score. While these pre-trained models are adept at understanding the general behaviors of code, they neither deal with the noise (e.g., some unrelated variable), nor depict the correlation between the code and the warning information. Additionally, these models break the identifiers down into sub-tokens, which makes it hard to capture the dependencies between those elements. This result is consistent with previous studies that acknowledged the limited advantage of pre-trained models in defect prediction [54].

FRUGAL, as a semi-supervised learning approach, previously demonstrated comparable performance to supervised feature-based approaches in actionable warning identification. However, its effectiveness significantly diminishes in the bug-sensitive warning identification, achieving only a 10.36% F1-score. One reason is the inherent challenge FRUGAL faces with extremely imbalanced datasets. In such scenarios, models may develop a bias towards the class with higher weight due to its more abundant information, leading to subpar predictions for the minority class [55], [56], [57]. In our experiments, to improve the F1-score for bug-sensitive warning identification, the model tended to over-predict bug-sensitive warnings. This approach increased recall but adversely affected precision, this is why FRUGAL achieved a high recall of 85.53% but a very low precision of 5.62%. It tended to classify most warnings as bug-sensitive warnings which is in vain for the purpose of reducing manual inspection costs.

Additionally, this strategy also resulted in a decreased recall for bug-insensitive warnings. Another contributing factor to FRUGAL’s under-performance in finer granularity tasks is the potential inappropriateness of its original features for the nuanced requirements of bug-sensitive warning identification.

LLM4SA, which leverages a language model-based approach, demonstrates potential in identifying bug warnings within static analysis results. However, it falls short in accurately detecting bug-sensitive warnings, achieving an F1-score of only 13.28%. Similar to FRUGAL, LLM4SA also tends to classify most warnings as bug-sensitive warnings, leading to a high recall of 83.47% but a very low precision of 7.21%, which is ineffective for reducing manual inspection costs. It is also important to note the considerable cost associated with this

approach: our experiments with GPT-3.5-turbo-0125 incurred expenses exceeding \$100 per run. Even when using open-source models such as DeepSeek-V3, which also achieved a lower F1-score of 11.55%, substantial computational resources were required (e.g., $8 \times$ H100 GPUs). In contrast, DeepFWI not only outperforms the LLM-based approach in terms of F1-score, but can also be efficiently executed on a single consumer-grade GPU (e.g., NVIDIA 3090), resulting in significantly lower hardware and financial costs. This comparison highlights the practical advantages of DeepFWI for scalable and cost-effective deployment.

Additionally, the limited effectiveness of LLM4SA can largely be attributed to the “hallucination problem [58],” a common challenge with language models where they generate plausible but incorrect or irrelevant responses. This issue is exacerbated in the complex and nuanced realm of bug semantics found in real-world datasets [59], [60]. LLM4SA solely relies on inputting code contexts and prompts to identify warnings. Due to the complexity of bug semantics in the real world, LLMs without fine-tuning struggle to capture these intricate bug semantics. Therefore, it fails to identify bug-sensitive warnings in real-world scenarios. This aligns with the transition from the artificial Juliet Test Suite to real-world datasets results in the real-world projects in [17]. Additionally, LLMs are significantly influenced by prompts. In Wen et al.’s approach, prompts tend to guide the model to perceive warnings as true, leading LLM4SA to classify most warnings as real bugs. Similarly, existing studies have shown that LLM models can produce erroneous results in bug detection, necessitating further verification through static analysis (SA) technologies [61].

Regarding basic alternatives, their performance still falls short of adequacy. The transformer model, lacking pre-training, faces challenges in learning code semantics and interpreting warning messages. This difficulty in training resulted in a sub-optimal performance, achieving only a 51.51% F1-score. The n-gram approach, another baseline, exhibits average performance, yet a little lower than “GHOST2”. This suggests that while the n-gram model can capture related patterns, its performance is hampered by noise and inability to learn deep semantics. We also find that as the base component of FineWAVE, the LSTM model outperformed the pre-trained models. This can be partly attributed to its strict orderliness of encoding, which makes it easier to learn the naturalness of source code, thereby acquiring more sequential dependencies. Therefore, their performance was not as good as that of the LSTM.

Additionally, in order to further present the improvement of the effectiveness of DeepFWI, we also conducted a paired t-test on the F1-score of DeepFWI and the baseline models. The results show that DeepFWI significantly outperforms all the baseline models with a p-value of 0.0049. This indicates that the improvement of DeepFWI is statistically significant. Furthermore, the improvement of DeepFWI is quite outperforming compared to some other works in the same field of software engineering [16], [17], [62], [63], [64]. In conclusion, DeepFWI considers more elements (including field code and warnings), integrates the function slicing to obtain rich while precise context and takes into account the interaction between warning

TABLE IV
EFFECTIVENESS OF THE MAIN COMPONENTS OF DEEPFWI

Components	Precision (%)	Recall (%)	F1-score (%)
W/O Field Code	71.08	57.66	63.67
W/O Warning-Aware Slice	70.88	61.58	65.90
W/O Cross-Attention	66.51	64.38	65.43
W/O Function Code	73.56	59.34	65.69
W/O RULE	71.05	62.19	66.32
W/O CATEGORY	72.71	61.68	66.74
W/O RANK	71.16	62.04	66.29
W/O CONFIDENCE	70.99	62.39	66.41
W/O Focal Loss	70.04	61.63	65.57
LSTM→Transformer	67.30	59.39	63.10
DeepFWI	75.52	60.31	67.06

messages and code. These enhancements allow DeepFWI to outperform the baseline models, underscoring its effectiveness in identifying warnings.

Answer to RQ1: Compared with the ten baseline models, DeepFWI demonstrated superior performance, achieving improvements of at least 9.03% in precision, and 13.17% in F1-score. These results underscore the effectiveness of DeepFWI in identifying bug-sensitive warnings.

2) RQ2: How effective are the individual components of DeepFWI?: In this RQ, we explore the impact of the key components of DeepFWI on its overall effectiveness. Our methodology primarily incorporates four components: warning-aware slicing, field code, cross-attention mechanism, and focal loss. To evaluate the significance of the individual components, we conducted an ablation study of DeepFWI by removing each component and observing the consequent effects on performance. Moreover, to further assess the impact of our design choices, we experimented by substituting the LSTM in our encoder with a transformer architecture.

The impact of each component of DeepFWI on its effectiveness is detailed in Table IV. Our first step was to remove the Field Code component, which resulted in a 3.39% decrease in the F1-score. This decline can be attributed to the diminished ability of the model to fully understand certain function code statements in the absence of the class variable segment provided by the Field Code. Consequently, this led to the model missing warnings that may contain bugs, resulting in false positives and inaccurate identification.

Upon removing the slicing component, a decline in DeepFWI’s effectiveness was observed, with a 1.16% decrease in the F1-score. Without this component, the model can encounter some noises (e.g. unrelated data flow in the same function), which adversely affects the tool’s focus on locating the bug information. Consequently, the precision of warning identification decreases by a large margin. Similarly, without the Cross-Attention component, there was a 1.63% decline in the F1-score, and notably, a substantial 9.01% drop in precision. As discussed earlier, the Cross-Attention mechanism is indeed important for facilitating the mutual integration of

warning messages and code. The results indicate that it is essential to consider precise semantics for fine-grained warning identification.

Furthermore, we evaluated the impact of excluding the Function Code component, which is responsible for providing the related information of the warning for the model. Removing this component led to a decrease in the F1-score by 1.37%. Specifically, removing Function Code resulted in a decrease in precision by 1.96%. The results suggest that Function Code provides an overall context, allowing the model to build a preliminary understanding of the program's general structure and semantics. In DeepFWI, while the function code sets the stage with broad context, the sliced IR zooms in on the critical areas that ultimately impact bug detection, thereby contributing to increased precision.

Additionally, we assessed the impact of excluding the RULE, CATEGORY, RANK, and CONFIDENCE components, which is the other warning information provided to the model. Removing these components led to a decrease in the F1-score by 0.74%, 0.32%, 0.77%, and 0.65%, respectively. Specifically, replacing these components with the warning message resulted in a decrease in precision by 4.47%, 2.81%, 4.36%, and 4.53%, respectively. The results suggest that these components provide additional information to the model, enabling it to build a more comprehensive understanding of the warning and focus on the potential bugs, thereby contributing to increased precision.

We also assessed the impact of excluding the Focal Loss component, which was employed to manage the significant imbalance in BSWarnings. Removing this component led to a modest decrease in the F1-score by 1.49%. Specifically, replacing Focal Loss with Cross-Entropy Loss resulted in a decrease in precision by 5.48%. The results suggest that Focal Loss's emphasis on less represented samples enhances the model's ability to gather more bug-related information, thereby contributing to increased precision.

Finally, when we substituted the LSTM encoder in DeepFWI with a Transformer model, contrary to expectations, there was a decrease in performance, dropping to 63.10% while we expect the self-attention mechanism of the Transformer to enhance DeepFWI's performance. The Transformer model processes tokens differently from LSTM, and this fundamental variance in semantic extraction could be the reason behind the diminished performance. Specifically, the self-attention mechanism of Transformers may focus on too many irrelevant dependencies, which makes LSTM a more suitable choice for our specific task.

Answer to RQ2: The effectiveness of each component of DeepFWI varies slightly, impacting performance by 0.32% to 3.96%. Field Code improves recall by broadening the context, while Warning-Aware Slicing and Cross-Attention enhance precision by focusing on more accurate semantics. Focal Loss, Function Code, the RULE, CATEGORY, RANK, and CONFIDENCE components slightly increases the F1-score and precision by better semantic extraction.

3) RQ3: What is the effectiveness of DeepFWI across different warning categories?: In this RQ, we analyze and discuss the

TABLE V
EFFECTIVENESS OF DEEPFWI ON EACH CATEGORY IN SPOTBUGS

Categories	Precision (%)	Recall (%)	F1-score (%)
<i>Bad Practice</i>	83.46	69.00	75.54
<i>Performance</i>	76.51	64.96	68.47
<i>Correctness</i>	68.68	67.57	68.12
<i>Malicious code</i>	76.15	57.70	65.65
<i>Dodgy code</i>	75.34	56.85	64.80
<i>Experimental</i>	66.67	62.50	64.52
<i>Internationalization</i>	71.28	53.17	60.91
<i>Multithreaded correctness</i>	60.53	57.50	58.97
<i>Security</i>	50.00	28.57	36.36
DeepFWI	75.52	60.31	67.06

effectiveness of DeepFWI in various categories of SpotBugs. Our classification comes from SpotBugs' categorization of its detection rules. There are a total of 10 types of categories in SpotBugs, i.e., *Correctness* (apparent coding mistakes), *Experimental* (experimental bug pattern), *Internationalization*, *Malicious code*, *Multithreaded correctness*, *Bogus random noise*, *Performance*, *Security*, and *Bad Practice*. Because the *Bogus Random Noise* category intended to be useful as a control in data mining experiments, not in finding actual bugs in software, we excluded it from our analysis.

As shown in Table V, DeepFWI achieves high performance in the *Bad Practice*, *Correctness*, and *Performance* categories, with overall F1-scores about 70%. *Bad Practice* warnings, often involving simpler bug patterns related to variable issues, are most effectively detected due to the model's focus on field code and variable-related semantics. In the *Correctness* category, which includes a variety of potential coding errors, our tool performs well, though some complex bugs needing more information may not be detected. The *Performance* category, similar to *Bad Practices*, shows good results, especially in identifying issues like unused code and inefficient loops. However, our tool might not always catch performance issues that are indirectly linked to bugs.

For *Security* category, DeepFWI performs worst on it, only achieving an F1-score of 36.36%. It is worth noting that there are very few bug-sensitive warnings in this category. BSWarnings contains a limited number of bug-sensitive warnings in the *Security* category, totaling only 63 instances. This scarcity of examples for the model to learn from could be a contributing factor to its low performance in validating issues within this specific category. This phenomenon also aligns with the result in the Fluffy on the classification of unexpected path-traversal flow [65]. Besides, accurately determining if these rules align with specific security issues in the code demands consideration of external factors, like inter-procedure dependencies and the execution environment.

For the other five categories, DeepFWI's performance hovers around a 60% F1-score, consistent with its overall effectiveness. Specifically, *Multithreaded Correctness* and *Experimental* exhibit similar levels of precision and recall which indicates that DeepFWI presents a balanced ability on them due to its components. The other three categories, however, show a stronger performance in precision. This pattern suggests that

while DeepFWI is adept at the detailed analysis of code and warnings, it sometimes misses nuances related to specific bug dependencies. The results across these categories reflect the tool's capability in handling a variety of code issues, though it also highlights areas for potential refinement in targeting bug-specific dependencies.

Answer to RQ3: *DeepFWI performs differently across various categories, with F1-scores ranging from 36.36% to 75.54%, precision ranging from 50.00% to 83.46%, and recall ranging from 28.57% to 69.00%. Among them, our model performs best in the Bad Practice category, and worst in the Security category mainly due to a small number of bug-sensitive warnings for it.*

4) **RQ4:** *Can our tool generalize when applied to real-world scenarios, and can it help developers identify new bugs?:* In this RQ, we aim to evaluate the effectiveness in real-world scenarios from the perspective of a developer. Specifically, we aim to explore the effectiveness of DeepFWI in practical scenarios instead just on our datasets. We discuss how our model's generalization in the latest (unseen) versions of projects, and whether it can identify (in)valid warnings. Due to the fact that the latest version does not have a fixed version for bugs, i.e., there is no ground truth for the latest version, we cannot automatically evaluate the performance of DeepFWI in the latest version. However, we can still evaluate the effectiveness of DeepFWI in the latest version by manually checking the warnings. Hence, we applied DeepFWI to 4 popular OSS projects which have over 1.5k stars, i.e., itext7, metadata-extractor, poi, and pdfbox.

Initially, we compiled the code and used SpotBugs to scan them, respectively. By analyzing the SpotBugs reports, we totally obtained 7,235 warnings from 4 projects.

Then, we applied DeepFWI on these warnings to filter out the false positives (i.e. bug-insensitive warnings). In this step, there remained 551 warnings (7.62%) in all projects. Based on these filtered warnings, we performed a manual check on each warning. For each warning, we not only analyze the warning's type and information to determine if the reported line is the root cause, but we also search for corresponding call issues based on the clues provided by each warning. This allows us to confirm whether the warning is part of the triggering chain of certain bugs. Finally, we totally found 25 bugs in the warnings validated by DeepFWI. We have submitted these bugs to the relevant developers and 4 of them have been confirmed and fixed by developers. In order to explore whether there may be a potential bug in the remaining warnings, we also conducted a sample analysis of the remaining 6,684 warnings. We randomly selected 200 warnings and manually checked them. Finally, we did not find any bugs in these warnings. Hence, we indicate that the remaining warnings are most bug-insensitive warnings.

Case Study: As shown in the Fig. 5, after filtering through our tool, we obtained two warnings: "Found reliance on default encoding in lines 5 and 6". We manually analyzed these two warnings and conducted an analysis of the entire function. We found that line 5 and line 6 could potentially lead to problems if the byte arrays contain data that is not compatible with the default charset. If the bytes do not represent valid characters in the

```

1 public void nextValidToken() throws java.io.IOException {
2     ...
3     assert n2 != null;
4     type = TokenType.Obj;
5     reference = Integer.parseInt(new String(n1));
6     generation = Integer.parseInt(new String(n2));
7     return;
8     ...
9 }

```

Fig. 5. A simplified bug example of nextValidToken method in itext7.

default charset, the resulting string could contain unexpected or incorrect characters, which could then cause `Integer.parseInt()` to throw a "NumberFormatException". To avoid this potential issue, it is generally recommended to specify a charset when converting bytes to a string. We have reported this issue to the developers, and now it employs an error handling mechanism to ensure that the byte arrays are properly converted to strings using a specified charset, thereby preventing potential errors related to character encoding.

Answer to RQ4: *When applying DeepFWI on four OSS projects with over 1.5k stars (i.e., itext7, metadata-extractor, poi, pdfbox), DeepFWI can help remove 92.38% of the warnings with potential false alarms, and enable us to discover 25 new bugs in these projects, 4 of which have been confirmed and fixed by developers.*

V. DISCUSSION

A. Threats to Validity

1) **External Validity:** As detailed in § III-B, our data collection process involved identifying bug-related keywords in commit messages from GitHub's projects. However, due to potential inaccuracies in commit descriptions, some commits may not be bug fixes or may contain unrelated changes, impacting BSWarnings's quality. Furthermore, the process of collecting warnings may also involve some poor-quality warnings which are not related to the specific bug. Hence, we manually sampled 384 warnings to assess BSWarnings, finding an accuracy rate of 92.60% for bug-sensitive warnings and 96.36% for bug-insensitive warnings, with a 95% confidence level.

2) **Internal Validity:** A potential threat to internal validity refers to the generalizability across ASATs. Following the existing research focusing on single ASATs [9], [12], [16], DeepFWI was initially evaluated using SpotBugs. To mitigate this threat and enhance the generalizability of our approach, DeepFWI is intentionally designed to depend only on the warning information which is common across different ASATs, such as the warning message [25], [66], [67]. Additionally, even though the tools do not have some specific warning attributes, such as RANK, they can still be adapted to our approach by simply ignoring these attributes, as they are not essential for the core functionality of DeepFWI as shown in the Section IV-D2. This strategic decision ensures that our approach can be broadly applicable and effective across a range of ASATs, facilitating its direct adaptation to other tools. Moving forward, our future work will expand to include a wider variety of ASATs, further broadening the scope and applicability of our approach.

Another potential threat to internal validity is the generalizability of the evaluation in Section IV-D4. In this evaluation, we manually checked the warnings generated by DeepFWI in the latest version of four popular open-source projects. However, our findings are limited to these specific projects. To address this, we selected widely used and highly starred projects on GitHub to cover a broad range of real-world scenarios. In addition, they are not part of the training set, which helps ensure that our model is not biased towards these projects. However, it is important to note that the results may not generalize to all projects.

Furthermore, a threat to internal validity relates to the manual inspection process and the use of stratified random sampling. Since determining whether a warning corresponds to a real bug requires human judgment, the manual inspection is inherently subjective and susceptible to human error or bias. To mitigate these threats, the manual inspection was conducted carefully by three senior developers, and any disagreements were resolved through discussion to reach a consensus. Additionally, the sample size was calculated to achieve a high confidence level, ensuring the statistical reliability of our findings.

It is worth noting that we identified 25 real bugs among the 551 warnings retained by DeepFWI. In practice, a single bug may trigger multiple warnings, and the vast majority of static analysis warnings do not correspond to real bugs, making the identification of true bugs highly challenging. Our approach aims to maximize bug discovery by retaining all warnings related to each potential bug, rather than restricting to just one warning per bug. As a result, while the absolute precision is not very high, the concentration of true bugs among the filtered warnings is significantly higher than in the original set of 7,235 warnings produced by the static analysis tool. Additionally, there may be some bugs that were not detected during manual inspection. To further assess the reliability of our filtering, we conducted a sample analysis of the warnings filtered out as bug-insensitive. This sampling confirmed that the vast majority of these filtered warnings were indeed not related to real bugs, supporting the effectiveness of our filtering process.

B. Limitations

Our current instantiation of DeepFWI targets Java projects analyzed by SpotBugs. Applying the approach to other programming languages (e.g., C/C++, C#) or different ASATs would require adapting the parsing front-end, intermediate representation (e.g., LLVM IR or CIL instead of Jimple), and warning attributes, and then retraining on language-specific datasets. Moreover, the adaptation to another language also requires the generation of a dataset like BSWarnings for the respective language. In addition, coding conventions and project-specific styles (e.g., use of fluent APIs, frameworks) may influence how informative warning messages and slices are. While the core idea of jointly modeling code, warning messages, and warning attributes is general, we cannot claim that the trained model will transfer directly to other languages or tools without such adaptation. We explicitly leave cross-language generalization

and handling diverse coding standards as important directions for future work.

VI. RELATED WORK

A. Learning-Based Approaches for Warning Identification

Most existing work on reducing false alarms focuses on using features involved in many aspects of warnings, code, and history information to predict warnings. Muske et al. provided a systematic survey of the post-processing of the warnings of ASATs [68]. For example, Heckman et al. employed 51 features and evaluated 15 machine learning approaches to achieve high accuracy [10]. Kim et al. used software change history to prioritize warning categories and proposed an algorithm for warning category prioritizing [5], [6]. Liang et al. proposed 3 impact factors as features on a new heuristic for identifying actionable warnings [69]. Ruthruff et al. applied a screening approach on existing logistic regression models to achieve a cost-effectively built prediction model to predict actionable warnings. Hanam et al. proposed an approach to distinguish actionable warnings by finding alerts with similar code patterns [13]. Based on these approaches and features, Wang et al. proposed a “Golden Feature” of 23 key attributes to achieve an AUC over 0.95 using linear SVM [9], [12]. However, Kang et al. found that there are leaked features and data duplication in the data, which can lead to a sudden drop in effectiveness [2]. Moreover, Tu et al. tried to unlock the semi-supervised approach for actionable warning identification. They proposed FRUGAL, a tuned semi-supervised approach that executes on a simple optimization policy and outperforms the supervised learning approaches for actionable warning identification. In addition, there is also another study to verify warnings at a function level. Kharkar et al. proposed a transformer-based learning approach to identify the false alarm on ASATs Infer [16]. Our work differs from the above studies by focusing on a finer granularity and considering multi-modal semantics with deep learning.

Recently, Wen et al. [17] conducted a cutting-edge study using LLMs to investigate whether LLMs can serve as human experts in identifying ASAT warnings. However, their focus was limited to six types of bugs. Additionally, due to the high volume of warnings and thus the considerable time and financial expense associated with using models like ChatGPT/LLama2-70B (e.g., 30s/warning for LLama2), these approaches may not be widely applicable to the large number of warnings typically encountered.

B. Reanalysis-Based Approaches for Improving ASATs

There are also studies working toward a different direction by improving the underlying analysis of ASATs. Many studies have attempted to reduce false positives in static analysis tools by focusing on pruning unreachable paths of a program. For example, Junker et al. proposed an approach that treats static analysis as a model-checking problem and uses SMT-solvers to prune infeasible paths, reducing false positive warnings [70]. Nguyen et al. applied deductive verification to eliminate false alarms [71], [72]. Li et al. introduced LLift, a framework

that combines static analysis and LLMs to identify use-before-initialization (UBI) bugs within the Linux kernel [73]. Ziyang Li et al. proposed a approach that combines static analysis with symbolic execution to enhance the contextual analysis alleviating needs for human specifications and inspection [74].

Additionally, some approaches employ dynamic analysis to enhance ASATs. For instance, Li et al. proposed residual investigation, a dynamic analysis technique to predict errors in executions and assess the validity of warnings [75]. Csallner et al. combined ASATs with concrete test-case generation to identify real warnings [76]. Busse et al. combined ASATs with dynamic symbolic execution but found that the traces provided by ASATs do not effectively guide search [77]. Kallingal et al. proposed a syntactic patching algorithm to provide executable code for further testing [78]. Murali et al. introduced FuzzS-lice, a framework that automatically prunes possible false positives among ASATs' warnings [79]. Furthermore, some studies focus on simplifying manual inspection. These approaches aim to screen warnings through semi-automated inspection [80], [81], feedback-based pruning [82], and checklist-based inspection [83].

VII. CONCLUSION

In this paper, we formulate the problem of warning identification at a fine-grained level, focusing on bug-sensitive warnings. To this end, we constructed BSWarnings, a dataset consisting of 280,273 static analysis warnings. Of these, 20,100 warnings are labeled as bug-sensitive, with the labeling process based on a combination of automated classification and validation through random sampling. To the best of our knowledge, this is the largest collection of data on warnings, ten times larger than the existing one [12]. Then, we proposed DeepFWI, a novel LSTM-based model for fine-grained warning identification of bugs for ASATs. DeepFWI employs a multi-faceted representation which captures fine-grained semantics of source code and warnings from ASATs and focuses more on their correlations with cross-attention. With an extensive experiment, DeepFWI shows an F1-score of 97.79% for reducing false alarms and 67.06% for confirming effective warnings. Compared with baselines, DeepFWI outperforms 9.03% in precision, and 13.17% in F1-score. Moreover, we employed DeepFWI on four real-world projects to filter out about 92% of warnings and then found 25 new bugs with minimal manual effort. We have released our data on our website [24] to replicate the results of this work and encourage further research.

ACKNOWLEDGMENT

We thank the Associate Editor and the anonymous reviewers for their insightful comments and constructive suggestions.

REFERENCES

- [1] R. Yedida, H. J. Kang, H. Tu, X. Yang, D. Lo, and T. Menzies, "How to find actionable static analysis warnings: A case study with findbugs," *IEEE Trans. Softw. Eng.*, vol. 49, no. 4, pp. 2856–2872, Apr. 2023.
- [2] H. J. Kang, K. L. Aw, and D. Lo, "Detecting false alarms from automatic static analysis tools: How far are we?" in *Proc. 44th Int. Conf. Softw. Eng. (ICSE)*. New York, NY, USA: Assoc. Comput. Machinery, 2022, pp. 698–709, doi: 10.1145/3510003.3510214.
- [3] L. Han et al., "A comprehensive study on quality assurance tools for Java," in *Proc. 32nd ACM SIGSOFT Int. Symp. Softw. Testing Anal. (ISSTA)*. New York, NY, USA: ACM, 2023, pp. 285–297.
- [4] L. Wei, Y. Liu, and S.-C. Cheung, "OASIS: Prioritizing static analysis warnings for android apps based on app user reviews," in *Proc. 11th Joint Meeting Found. Softw. Eng.*, 2017, pp. 672–682.
- [5] S. Kim and M. D. Ernst, "Prioritizing warning categories by analyzing software history," in *4th Int. Workshop Mining Softw. Repositories (MSR'07: ICSE Workshops 2007)*, Minneapolis, MN, USA. Piscataway, NJ, USA: IEEE Press, 2007, pp. 27–27.
- [6] S. Kim and M. D. Ernst, "Which warnings should I fix first?" in *Proc. 6th Joint Meeting Eur. Softw. Eng. Conf. ACM SIGSOFT Symp. Found. Softw. Eng.*, 2007, pp. 45–54.
- [7] Y. JunG, J. Kim, J. Shin, and K. Yi, "Taming false alarms from a domain-unaware C analyzer by a Bayesian statistical post analysis," in *Proc. Int. Static Anal. Symp.* Berlin, Germany: Springer, 2005, pp. 203–217.
- [8] J. R. Ruthruff, J. Penix, J. D. Morgenthaler, S. Elbaum, and G. Rothermel, "Predicting accurate and actionable static analysis warnings: An experimental approach," in *Proc. 30th Int. Conf. Softw. Eng. (ICSE)*. New York, NY, USA: ACM, 2008, pp. 341–350, doi: 10.1145/1368088.1368135.
- [9] X. Yang, J. Chen, R. Yedida, Z. Yu, and T. Menzies, "Learning to recognize actionable static code warnings (is intrinsically easy)," *Empirical Softw. Eng.*, vol. 26, no. 3, May 2021, doi: 10.1007/s10664-021-09948-6.
- [10] S. Heckman and L. Williams, "A model building process for identifying actionable static analysis alerts," in *Proc. Int. Conf. Softw. Testing Verification Validation*, 2009, pp. 161–170.
- [11] H. Sarah and W. Laurie, "A systematic literature review of actionable alert identification techniques for automated static code analysis," *Inf. Softw. Technol.*, vol. 53, no. 4, pp. 363–387, 2011, special section: Software Engineering track of the 24th Annual Symposium on Applied Computing. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0950584910002235>
- [12] J. Wang, S. Wang, and Q. Wang, "Is there a "golden" feature set for static warning identification? An experimental evaluation," in *Proc. 12th ACM/IEEE Int. Symp. Empirical Softw. Eng. Meas. (ESEM)*. New York, NY, USA: ACM, 2018, pp. 1–10.
- [13] Q. Hanam, L. Tan, R. Holmes, P. and Lam, L., "Finding patterns in static analysis alerts: Improving actionable alert ranking," in *Proc. 11th Work. Conf. Mining Softw. Repositories (MSR)*. New York, NY, USA: Assoc. Comput. Machinery, 2014, pp. 152–161, doi: 10.1145/2597073.2597100.
- [14] C. Williams and J. Hollingsworth, "Automatic mining of source code repositories to improve bug finding techniques," *IEEE Trans. Softw. Eng.*, vol. 31, no. 6, pp. 466–480, Jun. 2005.
- [15] U. Yüksel and H. Sözer, "Automated classification of static code analysis alerts: A case study," in *Proc. IEEE Int. Conf. Softw. Maintenance, Eindhoven, Netherlands*, 2013, pp. 532–535.
- [16] A. Kharkar et al., "Learning to reduce false positives in analytic bug detectors," in *Proc. 44th Int. Conf. Softw. Eng. (ICSE)*. New York, NY, USA: ACM, 2022, pp. 1307–1316, doi: 10.1145/3510003.3510153.
- [17] C. Wen et al., "Automatically inspecting thousands of static bug warnings with large language model: How far are we?" *ACM Trans. Knowl. Discov. Data*, vol. 18, no. 7, pp. 1–34, Mar. 2024, doi: 10.1145/3653718.
- [18] H. Tu and T. Menzies, "FRUGAL: Unlocking semi-supervised learning for software analytics," in *Proc. 36th IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, Melbourne, Australia, 2021, pp. 394–406.
- [19] Spotbugs, "SpotBugs," 2022. Accessed: Aug. 1, 2024. [Online]. Available: <https://spotbugs.github.io>
- [20] D. Guo et al., "GraphCodeBERT: Pre-training code representations with data flow," 2020, *arXiv:2009.08366*.
- [21] A. Vaswani et al., "Attention is all you need," *Adv. Neural Inf. Process. Syst.*, vol. 30, pp. 5998–6008, 2017.
- [22] M. Schuster and K. Paliwal, "Bidirectional recurrent neural networks," *IEEE Trans. Signal Process.*, vol. 45, no. 11, pp. 2673–2681, Nov. 1997.
- [23] P. F. Brown, V. J. Della Pietra, P. V. Desouza, J. C. Lai, and R. L. Mercer, "Class-based n -gram models of natural language," *Comput. Linguistics*, vol. 18, no. 4, pp. 467–480, 1992.
- [24] Website of This Work, "DeepFWI: Fine-grained static analysis warning identification using deep learning," 2024. Accessed Aug. 1, 2024. [Online]. Available: <https://sites.google.com/view/deepfwi/home>
- [25] SonarSource, "SonarQube," 2022. Accessed Aug. 1, 2024. [Online]. Available: <https://www.sonarqube.org>

- [26] D. Hovemeyer and W. Pugh, "Finding bugs is easy," *SIGPLAN Not.*, vol. 39, no. 12, pp. 92–106, Dec. 2004, doi: 10.1145/1052883.1052895.
- [27] C. Calcagno et al., "Moving fast with software verification," in *NASA Formal Methods*, K. Havelund, G. Holzmann, and R. Joshi, Eds. Cham: Springer Int. Publishing, 2015, pp. 3–11.
- [28] V. Balachandran, "Reducing human effort and improving quality in peer code reviews using automatic static analysis and reviewer recommendation," in *Proc. 35th Int. Conf. Softw. Eng. (ICSE)*, San Francisco, CA, USA, 2013, pp. 931–940.
- [29] H. Zhong, X. Wang, and H. Mei, "Inferring bug signatures to detect real bugs," *IEEE Trans. Softw. Eng.*, vol. 48, no. 2, pp. 571–584, Feb. 2022.
- [30] L. N. Q. Do, J. R. Wright, and K. Ali, "Why do software developers use static analysis tools? A user-centered study of developer needs and motivations," *IEEE Trans. Softw. Eng.*, vol. 48, no. 3, pp. 835–847, Mar. 2022.
- [31] D. Zhan, X. Yu, H. Zhang, and L. Ye, "ErrHunter: Detecting error-handling bugs in the Linux Kernel through systematic static analysis," *IEEE Trans. Softw. Eng.*, vol. 49, no. 2, pp. 684–698, Feb. 2023.
- [32] B. Johnson, Y. Song, E. Murphy-Hill, and R. Bowdidge, "Why don't software developers use static analysis tools to find bugs?" in *Proc. 35th Int. Conf. Softw. Eng. (ICSE)*, San Francisco, CA, USA, 2013, pp. 672–681.
- [33] P. Bielik, V. Raychev, and M. Vechev, "Learning a static analyzer from data," in *Proc. Comput. Aided Verification: 29th Int. Conf., CAV 2017, Heidelberg, Germany, July 24–28, 2017, Proceedings, Part I*. Berlin, Germany: Springer, 2017, pp. 233–253.
- [34] N. Ayewah, W. Pugh, D. Hovemeyer, J. D. Morgenthaler, and J. Penix, "Using static analysis to find bugs," *IEEE Softw.*, vol. 25, no. 5, pp. 22–29, Sep./Oct. 2008.
- [35] A. E. Hassan, "Automated classification of change messages in open source projects," in *Proc. ACM Symp. Appl. Comput. (SAC)*. New York, NY, USA: ACM, 2008, pp. 837–841, doi: 10.1145/1363686.1363876.
- [36] T. Parr and S. Harwell, "ANTLR 4," 2020. Accessed Aug. 1, 2024. [Online]. Available: <https://www.antlr.org/>
- [37] Oracle, "Oracle Java documentation," 2022. Accessed: Aug. 1, 2024. [Online]. Available: <https://docs.oracle.com/javase/tutorial/java/javaOO/variables.html>
- [38] Soot - a framework for analyzing and transforming Java and android applications," 2024. Accessed: Aug. 1, 2024. [Online]. Available: <https://soot-oss.github.io/soot/>
- [39] Z. Li et al., "VulDeePecker: A deep learning-based system for vulnerability detection," 2018, *arXiv:1801.01681*.
- [40] D. Zou, S. Wang, S. Xu, Z. Li, and H. Jin, "VulDeePecker: A deep learning-based system for multiclass vulnerability detection," *IEEE Trans. Dependable Secure Comput.*, vol. 18, no. 5, pp. 2224–2236, Sep./Oct. 2021.
- [41] J. Deshmukh, K. M. Annervaz, S. Podder, S. Sengupta, and N. Dubash, "Towards accurate duplicate bug retrieval using deep learning techniques," in *Proc. IEEE Int. Conf. Softw. Maintenance Evolution (ICSME)*, Shanghai, China, 2017, pp. 115–124.
- [42] W. Zaremba and I. Sutskever, "Learning to execute," 2014, *arXiv:1410.4615*.
- [43] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2017, pp. 2980–2988.
- [44] H. Aoyama, "A study of stratified random sampling," *Ann. Inst. Stat. Math.*, vol. 6, no. 1, pp. 1–36, 1954.
- [45] J. R. Landis and G. G. Koch, "An application of hierarchical Kappa-type statistics in the assessment of majority agreement among multiple observers," *Biometrics*, vol. 33, no. 2, pp. 363–374, 1977.
- [46] D. P. Kingma, "Adam: A method for stochastic optimization," 2014, *arXiv:1412.6980*.
- [47] Z. Feng et al., "CodeBERT: A pre-trained model for programming and natural languages," 2020, *arXiv:2002.08155*.
- [48] DeepSeek-AI, "DeepSeek-v3 technical report," 2020, *arXiv:2412.19437*.
- [49] H. Wu et al., "Peculiar: Smart contract vulnerability detection based on crucial data flow graph and pre-training techniques," in *Proc. IEEE 32nd Int. Symp. Softw. Rel. Eng. (ISSRE)*. Piscataway, NJ, USA: IEEE Press, 2021, pp. 378–389.
- [50] C. Ni, K. Yang, X. Xia, D. Lo, X. Chen, and X. Yang, "Defect identification, categorization, and repair: Better together," 2022, *arXiv:2204.04856*.
- [51] M. Allamanis, H. Jackson-Flux, and M. Brockschmidt, "Self-supervised bug detection and repair," *Adv. Neural Inf. Process. Syst.*, vol. 34, pp. 27865–27876, 2021.
- [52] C. Pornprasit and C. K. Tantithamthavorn, "DeepLineDP: Towards a deep learning approach for line-level defect prediction," *IEEE Trans. Softw. Eng.*, vol. 49, no. 1, pp. 84–98, Jan. 2022.
- [53] J. Ramos et al., "Using tf-idf to determine word relevance in document queries," in *Proc. 1st Instructional Conf. Mach. Learning*, vol. 242, no. 1, pp. 29–48, 2003.
- [54] W. U. Ahmad, S. Chakraborty, B. Ray, and K.-W. Chang, "Unified pre-training for program understanding and generation," 2021, *arXiv:2103.06333*.
- [55] L.-Z. Guo and Y.-F. Li, "Class-imbalanced semi-supervised learning with adaptive thresholding," in *Proc. 39th Int. Conf. Mach. Learn.*, K. Chaudhuri, S. Jegelka, L. Song, C. Szepesvari, G. Niu, and S. Sabato, Eds., vol. 162. PMLR, 2022, pp. 8082–8094. [Online]. Available: <https://proceedings.mlr.press/v162/guo22e.html>
- [56] Y. Yang and Z. Xu, "Rethinking the value of labels for improving class-imbalanced learning," in *Proc. Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2020, vol. 33, pp. 19290–19301.
- [57] M. Perez-Ortiz, P. Tiño, R. Mantiuk, and C. Hervás-Martínez, "Exploiting synthetically generated data with semi-supervised learning for small and imbalanced datasets," *Proc. AAAI Conf. Artif. Intell.*, vol. 33, no. 01, pp. 4715–4722, 2019.
- [58] V. Rawte, A. Sheth, and A. Das, "A survey of hallucination in large foundation models," 2023, *arXiv:2309.05922*.
- [59] M. D. Purba, A. Ghosh, B. J. Radford, and B. Chu, "Software vulnerability detection using large language models," in *Proc. IEEE 34th Int. Symp. Softw. Rel. Eng. Workshops (ISSREW)*, 2023, pp. 112–119.
- [60] C. Fang et al., "Large language models for code analysis: Do {LLMs} really do their job?" in *Proc. 33rd USENIX Secur. Symp. (USENIX)*, 2024, pp. 829–846.
- [61] Y. Sun et al., "GPTScan: Detecting logic vulnerabilities in smart contracts by combining GPT with program analysis," in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng. (ICSE)*. New York, NY, USA: ACM, 2024, pp. 1–13.
- [62] J. Zhang et al., "Detecting condition-related bugs with control flow graph neural network," in *Proc. 32nd ACM SIGSOFT Int. Symp. Softw. Testing Anal. (ISSTA)*. New York, NY, USA: ACM, 2023, pp. 1370–1382, doi: 10.1145/3597926.3598142.
- [63] X. Cheng, H. Wang, J. Hua, G. Xu, and Y. Sui, "DeepWukong: Statically detecting software vulnerabilities using deep graph neural network," *ACM Trans. Softw. Eng. Methodol.*, vol. 30, no. 3, pp. 1–33, Apr. 2021, doi: 10.1145/3436877.
- [64] Z. Li, D. Zou, S. Xu, Z. Chen, Y. Zhu, and H. Jin, "VulDeeLocator: A deep learning-based fine-grained vulnerability detector," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 4, pp. 2821–2837, Jul./Aug. 2022.
- [65] Y. W. Chow, M. Schäfer, and M. Pradel, "Beware of the unexpected: Bimodal taint analysis," in *Proc. 32nd ACM SIGSOFT Int. Symp. Softw. Testing Anal. (ISSTA)*. New York, NY, USA: ACM, 2023, pp. 211–222, doi: 10.1145/3597926.3598050.
- [66] PMD, "PMD source code analyzer," 2022. Accessed: Nov. 10, 2022. [Online]. Available: <https://pmd.github.io>
- [67] E. Aftandilian, R. Sauciuc, S. Priya, and S. Krishnan, "Building useful program analysis tools using an extensible Java compiler," in *Proc. IEEE 12th Int. Work. Conf. Source Code Anal. Manipulation*, Riva del Garda, Italy, 2012, pp. 14–23.
- [68] T. Muske and A. Serebrenik, "Survey of approaches for postprocessing of static analysis alarms," *ACM Comput. Surv.*, vol. 55, no. 3, Feb. 2022, doi: 10.1145/3494521.
- [69] G. Liang, L. Wu, Q. Wu, Q. Wang, T. Xie, and H. Mei, "Automatic construction of an effective training set for prioritizing static analysis warnings," in *Proc. 25th IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*. New York, NY, USA: ACM, 2010, pp. 93–102, doi: 10.1145/1858996.1859013.
- [70] M. JunKer, R. Huuck, A. Fehnker, and A. Knapp, "SMT-based false positive elimination in static program analysis," in *Formal Methods and Software Engineering*, T. Aoki and K. Taguchi, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2012, pp. 316–331.
- [71] T.-T. Nguyen, T. Aoki, T. Tomita, and I. Yamada, "Multiple program analysis techniques enable precise check for SEI CERT C coding standard," in *Proc. 26th Asia-Pacific Softw. Eng. Conf. (APSEC)*. Putrajaya, Malaysia, Piscataway, NJ, USA: IEEE Press, 2019, pp. 70–77.

- [72] T. T. Nguyen, P. Malechuan, T. Aoki, T. Tomita, and I. Yamada, "Reducing false positives of static analysis for SEI CERT C coding standard," in *Proc. IEEE/ACM Joint 7th Int. Workshop Conducting Empirical Stud. Industry (CESI) 6th Int. Workshop Softw. Eng. Res. Ind. Pract. (SER&IP)*. Piscataway, NJ, USA: IEEE Press, 2019, pp. 41–48.
- [73] H. Li, Y. Hao, Y. Zhai, and Z. Qian, "Enhancing static analysis for practical bug detection: An LLM-integrated approach," *Proc. ACM Program. Lang.*, vol. 8, no. OOPSLA1, Apr. 2024, doi: 10.1145/3649828.
- [74] Z. Li, S. Dutta, and M. Naik, "IRIS: LLM-assisted static analysis for detecting security vulnerabilities," in *Proc. 13th Int. Conf. Learn. Represent.*, 2025, pp. 1–24.
- [75] K. Li, C. Reichenbach, C. Csallner, and Y. Smaragdakis, "Residual investigation: Predictive and precise bug detection," *ACM Trans. Softw. Eng. Method. (TOSEM)*, vol. 24, no. 2, pp. 1–32, 2014.
- [76] C. Csallner and Y. Smaragdakis, "Check'n'crash: Combining static checking and testing," in *Proc. 27th Int. Conf. Softw. Eng.*, 2005, pp. 422–431.
- [77] F. Busse, P. Gharat, C. Cadar, and A. F. Donaldson, "Combining static analysis error traces with dynamic symbolic execution (experience paper)," in *Proc. 31st ACM SIGSOFT Int. Symp. Softw. Testing Anal. (ISSTA)*. New York, NY, USA: ACM, 2022, pp. 568–579, doi: 10.1145/3533767.3534384.
- [78] A. Kallingal Joshy, X. Chen, B. Steenhoeck, and W. Le, "Validating static warnings via testing code fragments," in *Proc. 30th ACM SIGSOFT Int. Symp. Softw. Testing Anal. (ISSTA)*. New York, NY, USA: ACM, 2021, pp. 540–552, doi: 10.1145/3460319.3464832.
- [79] A. Murali, N. Mathews, M. Alfadel, M. Nagappan, and M. Xu, "FuzzSlice: Pruning false positives in static analysis warnings through function-level fuzzing," in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng. (ICSE)*. New York, NY, USA: ACM, 2024, doi: 10.1145/3597503.3623321.
- [80] X. Rival, "Abstract dependences for alarm diagnosis," in *Proc. Program. Lang. Syst. Third Asian Symp., APLAS 2005, Tsukuba, Japan*. Berlin, Germany: Springer, 2005, pp. 347–363.
- [81] X. Rival, "Understanding the origin of alarms in ASTRÉE," in *Proc. Static Anal. 12th Int. Symp., SAS 2005, London, UK*. Berlin, Germany: Springer, 2005, pp. 303–319.
- [82] C. Sadowski, J. Van Gogh, C. Jaspan, E. Soderberg, and C. Winter, "Tricorder: Building a program analysis ecosystem," in *Proc. IEEE/ACM 37th IEEE Int. Conf. Softw. Eng.* Piscataway, NJ, USA: IEEE Press, 2015, vol. 1, pp. 598–608.
- [83] S. Panichella, V. Arnaoudova, M. Di Penta, and G. Antoniol, "Would static analysis tools help developers with code reviews?" in *Proc. IEEE 22nd Int. Conf. Softw. Anal., Evolution, Reengineering (SANER)*, Montreal, QC, Canada, 2015, pp. 161–170.